

Reconstruction Metric - GPU Optimized

Karina

2026-02-17

Table of contents

0.1	1. Setup e Imports	2
0.2	2. Cargar Datos	2
0.3	3. Cargar SAE y Extraer Features	2
0.4	4. Split Train/Test y Filtrar BSPs de Piezas	4
0.5	5. Funciones GPU-Optimizadas	4
0.6	6. Fase 1: Identificar Features de Alta Precisión (GPU)	5
0.7	7. Fase 2: Reconstruir Tableros en Test Set (GPU)	8
0.8	8. Análisis de Resultados	12
0.9	9. Guardar Resultados	15

```
# Configuración para visualización en PDF
import pandas as pd
import numpy as np
import sys
import os
from sae.tools.experiment_utils import get_metrics_dir

pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 100)
np.set_printoptions(linewidth=100, edgeitems=3)

os.environ['COLUMNS'] = '100'

print(" Configuración para PDF lista")
```

Configuración para PDF lista

```
NAMING_META = {
    'variante': 'standard',      # Arquitectura del SAE
    'tecnica': 'l1',             # L1 con penalty annealing
    'capa': 6,                   # Layer del modelo OthelloGPT
    'juegos': 1000,              # Número de partidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt', # Path base para checkpoints
    'experiment_base_path': os.path.abspath('.././experiments')      # sae/experiments/
}

print('\n Metadata de naming:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')
```

Metadata de naming:

variante: standard

tecnica: l1

capa: 6

juegos: 1000

base_path: C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt

experiment_base_path: c:\\Users\\Esposa\\Documents\\Repos\\othello_world\\sae\\experiments

0.1 1. Setup e Imports

```
import numpy as np
import torch
import sys
from pathlib import Path
from tqdm import tqdm
import time

project_root = Path('../..').resolve()
sys.path.insert(0, str(project_root))

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memoria: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
```

Device: cuda
GPU: NVIDIA GeForce RTX 5060
Memoria: 8.55 GB

0.2 2. Cargar Datos

```
# Cargar activaciones
activations_path = project_root / "sae" / "activations" / "data" / "layer5_200games.npy"
activations = np.load(activations_path)

print(f"Activaciones del modelo:")
print(f"  Shape: {activations.shape}")
print(f"  Memoria: {activations.nbytes / (1024**2):.2f} MB")
```

Activaciones del modelo:
 Shape: (11800, 512)
 Memoria: 23.05 MB

```
# Cargar ground truth de BSPs
bsp_gt_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.npy"
bsp_names_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.names.npy"

bsp_ground_truth = np.load(bsp_gt_path)
bsp_names = np.load(bsp_names_path, allow_pickle=True)

print(f"\nGround truth BSPs:")
print(f"  Shape: {bsp_ground_truth.shape}")
print(f"  Total BSPs: {len(bsp_names)}")
```

Ground truth BSPs:
 Shape: (11800, 198)
 Total BSPs: 198

0.3 3. Cargar SAE y Extraer Features

```

from sae.models.sae import SparseAutoencoder
from sae.tools.naming_utils import get_checkpoint_dir, get_model_name

input_dim = 512
hidden_dim = 16384

sae = SparseAutoencoder(input_dim, hidden_dim).to(device)
checkpoint_dir = get_checkpoint_dir(
    NAMING_META['base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa']
)
model_filename = get_model_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'best'
)
model_path = checkpoint_dir / model_filename
checkpoint = torch.load(model_path, map_location=device, weights_only=False)
sae.load_state_dict(checkpoint['model_state_dict'])
sae.eval()

# Extraer info de epoch/val_mse según formato del checkpoint
metadata = checkpoint.get('metadata', {})
history = checkpoint.get('history', {})
epoch = checkpoint.get('epoch', history.get('best_epoch', metadata.get('estado', 'N/A')))
val_mse = checkpoint.get('val_mse', history.get('best_val_mse', 'N/A'))

print(f"SAE cargado:")
print(f"  Input: {input_dim}, Hidden: {hidden_dim}")
print(f"  Expansion: {hidden_dim/input_dim}x")
print(f"  Epoch: {epoch}, Val MSE: {val_mse if val_mse == 'N/A' else f'{val_mse:.6f}'}")
print(f"  Claves checkpoint: {list(checkpoint.keys())}")

```

```

SAE cargado:
  Input: 512, Hidden: 16384
  Expansion: 32.0x
  Epoch: best, Val MSE: N/A
  Claves checkpoint: ['model_state_dict', 'config', 'metadata', 'history', 'optimizer_state_dict']

```

```

# Extraer features del SAE en GPU
print("Extrayendo features del SAE...")
activations_tensor = torch.from_numpy(activations).float().to(device)

with torch.no_grad():
    sae_features = torch.relu(sae.encoder(activations_tensor))

print(f"\nFeatures SAE:")
print(f"  Shape: {sae_features.shape}")
print(f"  Device: {sae_features.device}")
print(f"  Sparsity: {(sae_features == 0).float().mean():.2%}")
print(f"  Activaciones promedio: {(sae_features > 0).sum(dim=1).float().mean():.1f}")

```

Extrayendo features del SAE...

```

Features SAE:
  Shape: torch.Size([11800, 16384])
  Device: cuda:0

```

Sparsity: 96.54%
Activaciones promedio: 567.7

0.4 4. Split Train/Test y Filtrar BSPs de Piezas

```
# Split: primeras 100 partidas = train, últimas 100 = test
n_moves = 59
split_idx = 100 * n_moves

sae_features_train = sae_features[:split_idx]
sae_features_test = sae_features[split_idx:]

bsp_gt_train = bsp_ground_truth[:split_idx]
bsp_gt_test = bsp_ground_truth[split_idx:]

print(f"Train set: {sae_features_train.shape}")
print(f"Test set: {sae_features_test.shape}")
```

Train set: torch.Size([5900, 16384])
Test set: torch.Size([5900, 16384])

```
# Filtrar BSPs de piezas (terminan en '1' o '2', no en '0')
bsp_pieces_indices = []
bsp_pieces_names = []

for i, name in enumerate(bsp_names):
    if len(name) == 6 and name.startswith('BSP') and not name.endswith('0'):
        bsp_pieces_indices.append(i)
        bsp_pieces_names.append(name)

bsp_pieces_indices = np.array(bsp_pieces_indices)

# Aplicar filtro y convertir a tensores GPU
bsp_gt_train_pieces = torch.from_numpy(bsp_gt_train[:, bsp_pieces_indices]).bool().to(device)
bsp_gt_test_pieces = torch.from_numpy(bsp_gt_test[:, bsp_pieces_indices]).bool().to(device)

print(f"\nBSPs para Reconstruction:")
print(f"  Total BSPs de piezas: {len(bsp_pieces_indices)}")
print(f"  Train shape: {bsp_gt_train_pieces.shape}")
print(f"  Test shape: {bsp_gt_test_pieces.shape}")
```

BSPs para Reconstruction:
Total BSPs de piezas: 128
Train shape: torch.Size([5900, 128])
Test shape: torch.Size([5900, 128])

0.5 5. Funciones GPU-Optimizadas

```
def fast_precision_score_gpu(y_true, y_pred, eps=1e-8):
    """
    Precisión vectorizada en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions, n_candidates) booleano

    Returns:
```

```

        precision_scores: Tensor (n_candidates,)
    """
    y_true_expanded = y_true.unsqueeze(1)

    tp = (y_true_expanded & y_pred).sum(dim=0).float()
    fp = (~y_true_expanded & y_pred).sum(dim=0).float()

    precision = tp / (tp + fp + eps)

    return precision

def fast_f1_score_gpu(y_true, y_pred, eps=1e-8):
    """
    F1 score vectorizado en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions,) booleano

    Returns:
        f1: Float
    """
    tp = (y_true & y_pred).sum().float()
    fp = (~y_true & y_pred).sum().float()
    fn = (y_true & ~y_pred).sum().float()

    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)

    f1 = 2 * (precision * recall) / (precision + recall + eps)

    return f1.item()

print(" Funciones GPU definidas")

```

Funciones GPU definidas

0.6 6. Fase 1: Identificar Features de Alta Precisión (GPU)

Para cada BSP, encontrar features con precisión 0.95 en el train set.

```

def identify_high_precision_features_gpu(
    sae_features_train,
    bsp_gt_train,
    precision_threshold=0.95,
    thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0],
    batch_size=512,
    verbose=True
):
    """
    Identifica features con precisión threshold para cada BSP (GPU OPTIMIZADO).

    CORRECCIÓN: Solo guarda feature_idx (NO guarda thresholds especificos).
    El threshold se aplicará globalmente en la Fase 2.

    Args:
        sae_features_train: Tensor (n_train, n_features) en GPU
        bsp_gt_train: Tensor (n_train, n_bsps) en GPU, bool
        precision_threshold: Mínima precisión requerida (default 0.95)
        thresholds: Umbrales de activación a probar (para identificar features útiles)
    """

```

```

    batch_size: Features a procesar simultáneamente

Returns:
    high_precision_features: dict {bsp_idx: [feature_idx, ...]} ← Solo índices
"""
n_positions, n_features = sae_features_train.shape
n_bsps = bsp_gt_train.shape[1]

if verbose:
    print("="*60)
    print("FASE 1: IDENTIFICAR FEATURES DE ALTA PRECISIÓN")
    print("="*60)
    print(f"Threshold de precisión: {precision_threshold}")
    print(f"BSPs: {n_bsps}")
    print(f"Features: {n_features}")
    print(f"Batch size: {batch_size}")
    print("="*60)

# Pre-calcular máximos
f_max = sae_features_train.max(dim=0)[0]
active_features = (f_max > 0).nonzero(as_tuple=True)[0]
n_active = len(active_features)

if verbose:
    print(f"\nFeatures activas: {n_active}/{n_features} ({n_active/n_features*100:.1f}%)")
    print()

high_precision_features = {}
thresholds_tensor = torch.tensor(thresholds, device=sae_features_train.device)

pbar = tqdm(range(n_bsps), desc="Procesando BSPs") if verbose else range(n_bsps)

for bsp_idx in pbar:
    bsp_labels = bsp_gt_train[:, bsp_idx]

    # Skip si nunca está activa
    if not bsp_labels.any():
        high_precision_features[bsp_idx] = []
        continue

    hp_features_for_bsp = []
    features_found = torch.zeros(n_active, dtype=torch.bool, device=sae_features_train.device)

    # Procesar features activas en batches
    for batch_start in range(0, n_active, batch_size):
        batch_end = min(batch_start + batch_size, n_active)
        batch_features_idx = active_features[batch_start:batch_end]

        # Extraer features del batch
        batch_activations = sae_features_train[:, batch_features_idx]
        batch_f_max = f_max[batch_features_idx]

        # Probar cada threshold para IDENTIFICAR features útiles
        for t in thresholds_tensor:
            # Binarizar: (n_positions, batch_size)
            predictions = batch_activations > (t * batch_f_max.unsqueeze(0))

            # Solo calcular precisión si hay predicciones
            has_predictions = predictions.any(dim=0)

            if has_predictions.any():
                # Calcular precisión solo para features con predicciones

```

```

        precision_scores = torch.zeros(predictions.shape[1], device=sae_features_train.device)
        precision_scores[has_predictions] = fast_precision_score_gpu(
            bsp_labels,
            predictions[:, has_predictions]
        )

        # Encontrar features que cumplen threshold
        good_features = precision_scores >= precision_threshold

        if good_features.any():
            # CORRECCIÓN: Solo guardar feature_idx (NO thresholds)
            local_indices = good_features.nonzero(as_tuple=True)[0]
            for local_idx in local_indices:
                global_idx = batch_start + local_idx.item()
                if not features_found[global_idx]:
                    feature_idx = batch_features_idx[local_idx].item()
                    hp_features_for_bsp.append(feature_idx) # ← Solo índice
                    features_found[global_idx] = True

        high_precision_features[bsp_idx] = hp_features_for_bsp

    return high_precision_features

print(" Función Fase 1 definida (CORREGIDA: solo guarda feature_idx)")

```

Función Fase 1 definida (CORREGIDA: solo guarda feature_idx)

```

# Ejecutar Fase 1
start_time = time.time()

high_precision_features = identify_high_precision_features_gpu(
    sae_features_train,
    bsp_gt_train_pieces,
    precision_threshold=0.95,
    batch_size=512,
    verbose=True
)

phase1_time = time.time() - start_time

# Estadísticas
n_features_per_bsp = [len(features) for features in high_precision_features.values()]

print("\n" + "="*60)
print("RESULTADOS FASE 1")
print("="*60)
print(f"Total BSPs: {len(high_precision_features)}")
print(f"Features promedio por BSP: {np.mean(n_features_per_bsp):.1f}")
print(f"BSPs con 0 features: {np.sum(np.array(n_features_per_bsp) == 0)}")
print(f"BSPs con 1+ features: {np.sum(np.array(n_features_per_bsp) > 0)}")
print(f"BSPs con 5+ features: {np.sum(np.array(n_features_per_bsp) >= 5)}")
print(f"\nTiempo Fase 1: {phase1_time:.2f} seg")
print("="*60)

```

```

=====
FASE 1: IDENTIFICAR FEATURES DE ALTA PRECISIÓN
=====
Threshold de precisión: 0.95
BSPs: 128
Features: 16384
Batch size: 512

```

```
=====
Features activas: 7899/16384 (48.2%)
```

```
Procesando BSPs: 100%|      | 128/128 [02:08<00:00, 1.01s/it]
```

```
=====
RESULTADOS FASE 1
=====
```

```
Total BSPs: 128
Features promedio por BSP: 1920.8
BSPs con 0 features: 0
BSPs con 1+ features: 128
BSPs con 5+ features: 128
```

```
Tiempo Fase 1: 128.92 seg
=====
```

0.7 7. Fase 2: Reconstruir Tableros en Test Set (GPU)

```
def reconstruct_boards_gpu(
    sae_features_test,
    bsp_gt_test,
    high_precision_features,
    f_max,
    global_thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0],
    batch_size=1000,
    verbose=True
):
    """
    Reconstruye tableros usando BARRIDO GLOBAL de thresholds (GPU OPTIMIZADO).

    CORRECCIÓN: Prueba CADA threshold global t aplicado a TODAS las features,
    y reporta el mejor F1 entre todos los thresholds (Ecuación 5 del paper).

    Args:
        sae_features_test: Tensor (n_test, n_features) en GPU
        bsp_gt_test: Tensor (n_test, n_bsps) en GPU, bool
        high_precision_features: dict {bsp_idx: [feature_idx, ...]} ← Solo índices
        f_max: Tensor (n_features,) - máximos calculados del train set
        global_thresholds: Lista de thresholds a probar GLOBALMENTE
        batch_size: Posiciones a procesar simultáneamente por threshold

    Returns:
        reconstruction_score: Mejor F1 entre todos los thresholds
        best_threshold: Threshold óptimo
        f1_per_threshold: Array de F1 para cada threshold
    """
    n_positions, n_features = sae_features_test.shape
    n_bsps = bsp_gt_test.shape[1]
    device = sae_features_test.device

    if verbose:
        print("="*60)
        print("FASE 2: BARRIDO GLOBAL DE THRESHOLDS (GPU VECTORIZADO)")
        print("="*60)
        print(f"Posiciones: {n_positions}")
```



```

print(f"BSPs: {n_bsps}")
print(f"Thresholds a probar: {len(global_thresholds)}")
print(f"Batch size: {batch_size}")
print("="*60)
print()

# PRE-PROCESAR: Convertir high_precision_features a estructuras GPU eficientes
if verbose:
    print("Pre-procesando features de alta confianza...")

max_hp_features = max((len(feats) for feats in high_precision_features.values()), default=0)

# Matriz: (n_bsps, max_hp_features) con -1 para padding
hp_feature_indices = torch.full((n_bsps, max_hp_features), -1, dtype=torch.long, device=device)

for bsp_idx, hp_features in high_precision_features.items():
    for i, feat_idx in enumerate(hp_features):
        hp_feature_indices[bsp_idx, i] = feat_idx

if verbose:
    print(f" Pre-procesamiento completo")
    print(f" Max features por BSP: {max_hp_features}")
    print()

# =====
# BARRIDO GLOBAL: Probar cada threshold
# =====
f1_per_threshold = []

for t_global in tqdm(global_thresholds, desc="Barrido global de thresholds"):
    # Para este threshold global, reconstruir TODOS los tableros
    f1_scores_this_threshold = []

    n_batches = (n_positions + batch_size - 1) // batch_size

    for batch_idx in range(n_batches):
        start_idx = batch_idx * batch_size
        end_idx = min(start_idx + batch_size, n_positions)
        batch_n_positions = end_idx - start_idx

        # Features del batch: (batch_n_positions, n_features)
        batch_features = sae_features_test[start_idx:end_idx]
        batch_gt = bsp_gt_test[start_idx:end_idx]

        # Inicializar predicciones: (batch_n_positions, n_bsps)
        predictions = torch.zeros((batch_n_positions, n_bsps), dtype=torch.bool, device=device)

        # Para cada BSP, detectar si alguna feature de alta confianza se activa
        for bsp_idx in range(n_bsps):
            # Features de alta confianza para esta BSP
            feat_indices = hp_feature_indices[bsp_idx]

            # Filtrar padding (-1)
            valid_mask = feat_indices >= 0
            if not valid_mask.any():
                continue

            feat_indices = feat_indices[valid_mask]

            # Extraer activaciones: (batch_n_positions, n_valid_features)
            selected_features = batch_features[:, feat_indices]

```

```

        # CRÍTICO: Aplicar threshold GLOBAL
        scaled_thresholds = t_global * f_max[feat_indices]

        # Verificar si alguna feature supera el threshold global
        activated = selected_features > scaled_thresholds.unsqueeze(0)

        # Regla de reconstrucción: Si alguna feature se activa → predecir True
        predictions[:, bsp_idx] = activated.any(dim=1)

    # Calcular F1 para cada posición del batch
    for i in range(batch_n_positions):
        pred = predictions[i]
        gt = batch_gt[i]

        if pred.any() or gt.any():
            f1 = fast_f1_score_gpu(gt, pred)
        else:
            f1 = 1.0 # Tablero vacío correctamente predicho

        f1_scores_this_threshold.append(f1)

    # F1 promedio para este threshold
    avg_f1_this_threshold = np.mean(f1_scores_this_threshold)
    f1_per_threshold.append(avg_f1_this_threshold)

    if verbose:
        print(f"  t={t_global:.1f}: F1={avg_f1_this_threshold:.4f}")

# =====
# FASE 3: Reportar el mejor threshold
# =====
f1_per_threshold = np.array(f1_per_threshold)
best_threshold_idx = f1_per_threshold.argmax()
reconstruction_score = f1_per_threshold[best_threshold_idx]
best_threshold = global_thresholds[best_threshold_idx]

if verbose:
    print()
    print("="*60)
    print("MEJOR THRESHOLD GLOBAL")
    print("="*60)
    print(f"Threshold óptimo: t={best_threshold:.1f}")
    print(f"Reconstruction Score: {reconstruction_score:.4f}")
    print("="*60)

    return reconstruction_score, best_threshold, f1_per_threshold

print(" Función Fase 2 definida (CORREGIDA: barrido global de thresholds)")

```

Función Fase 2 definida (CORREGIDA: barrido global de thresholds)

```

# Ejecutar Fase 2 con Barrido Global de Thresholds
start_time = time.time()

# Usar f_max del train set
f_max_train = sae_features_train.max(dim=0)[0]

reconstruction_score, best_threshold, f1_per_threshold = reconstruct_boards_gpu(
    sae_features_test,
    bsp_gt_test_pieces,
    high_precision_features,

```

```

    f_max_train,
    global_thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0],
    batch_size=1000,
    verbose=True
)

phase2_time = time.time() - start_time
total_time = phase1_time + phase2_time

print("\n" + "="*60)
print("RESULTADO FINAL - BOARD RECONSTRUCTION")
print("="*60)
print(f"Reconstruction Score: {reconstruction_score:.4f}")
print(f"Mejor threshold global: t={best_threshold:.1f}")
print(f"Objetivo (paper): 0.95")
print(f"Diferencia: {reconstruction_score - 0.95:+.4f}")
print(f"\nTiempo Fase 2: {phase2_time:.2f} seg")
print(f"Tiempo Total: {total_time:.2f} seg ({total_time/60:.2f} min)")
print("="*60)

# Sanity check: t=1.0 debe dar F1 muy bajo
print("\n Sanity Check:")
print(f" F1 en t=0.0: {f1_per_threshold[0]:.4f}")
print(f" F1 en t=1.0: {f1_per_threshold[-1]:.4f} (debería ser < 0.2)")
if f1_per_threshold[-1] > 0.2:
    print(" WARNING: t=1.0 tiene F1 muy alto, revisar implementación")

```

```

=====
FASE 2: BARRIDO GLOBAL DE THRESHOLDS (GPU VECTORIZADO)
=====

```

```

Posiciones: 5900
BSPs: 128
Thresholds a probar: 11
Batch size: 1000
=====

```

```

Pre-procesando features de alta confianza...
Pre-procesamiento completo
Max features por BSP: 4630

```

```

Barrido global de thresholds:  9%|          | 1/11 [00:03<00:38,  3.81s/it]

```

```

    t=0.0: F1=0.4041

```

```

Barrido global de thresholds: 18%|          | 2/11 [00:07<00:34,  3.79s/it]

```

```

    t=0.1: F1=0.4152

```

```

Barrido global de thresholds: 27%|          | 3/11 [00:11<00:30,  3.76s/it]

```

```

    t=0.2: F1=0.4200

```

```

Barrido global de thresholds: 36%|          | 4/11 [00:15<00:26,  3.74s/it]

```

```

    t=0.3: F1=0.4274

```

```

Barrido global de thresholds: 45%|          | 5/11 [00:18<00:22,  3.74s/it]

```

```

    t=0.4: F1=0.4410

```

Barrido global de thresholds: 55%| | 6/11 [00:22<00:18, 3.72s/it]

t=0.5: F1=0.4503

Barrido global de thresholds: 64%| | 7/11 [00:26<00:14, 3.70s/it]

t=0.6: F1=0.4110

Barrido global de thresholds: 73%| | 8/11 [00:29<00:11, 3.71s/it]

t=0.7: F1=0.3073

Barrido global de thresholds: 82%| | 9/11 [00:33<00:07, 3.73s/it]

t=0.8: F1=0.1990

Barrido global de thresholds: 91%| | 10/11 [00:37<00:03, 3.81s/it]

t=0.9: F1=0.1298

Barrido global de thresholds: 100%| | 11/11 [00:41<00:00, 3.79s/it]

t=1.0: F1=0.0470

=====
MEJOR THRESHOLD GLOBAL

=====
Threshold óptimo: t=0.5
Reconstruction Score: 0.4503

=====
RESULTADO FINAL - BOARD RECONSTRUCTION

=====
Reconstruction Score: 0.4503
Mejor threshold global: t=0.5
Objetivo (paper): 0.95
Diferencia: -0.4997

Tiempo Fase 2: 46.65 seg
Tiempo Total: 175.57 seg (2.93 min)

=====
Sanity Check:
F1 en t=0.0: 0.4041
F1 en t=1.0: 0.0470 (debería ser < 0.2)

0.8 8. Análisis de Resultados

```
# Análisis del barrido de thresholds
import matplotlib.pyplot as plt

global_thresholds = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]

print("="*60)
print("BARRIDO DE THRESHOLDS - F1 POR THRESHOLD")
```

```

print("="*60)
for i, (t, f1) in enumerate(zip(global_thresholds, f1_per_threshold)):
    marker = " ← MEJOR" if i == f1_per_threshold.argmax() else ""
    print(f"t={t:.1f}: F1={f1:.4f}{marker}")
print("="*60)

# Plotear el barrido
plt.figure(figsize=(10, 6))
plt.plot(global_thresholds, f1_per_threshold, marker='o', linewidth=2, markersize=8)
plt.axhline(y=reconstruction_score, color='r', linestyle='--', label=f'Best F1={reconstruction_score:.4f}')
plt.axhline(y=0.95, color='g', linestyle=':', label='Target (paper)=0.95')
plt.xlabel('Threshold Global', fontsize=12)
plt.ylabel('F1 Score', fontsize=12)
plt.title('Board Reconstruction: Barrido Global de Thresholds', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print(f"\n Threshold óptimo: t={best_threshold:.1f}")
print(f" Reconstruction Score: {reconstruction_score:.4f}")

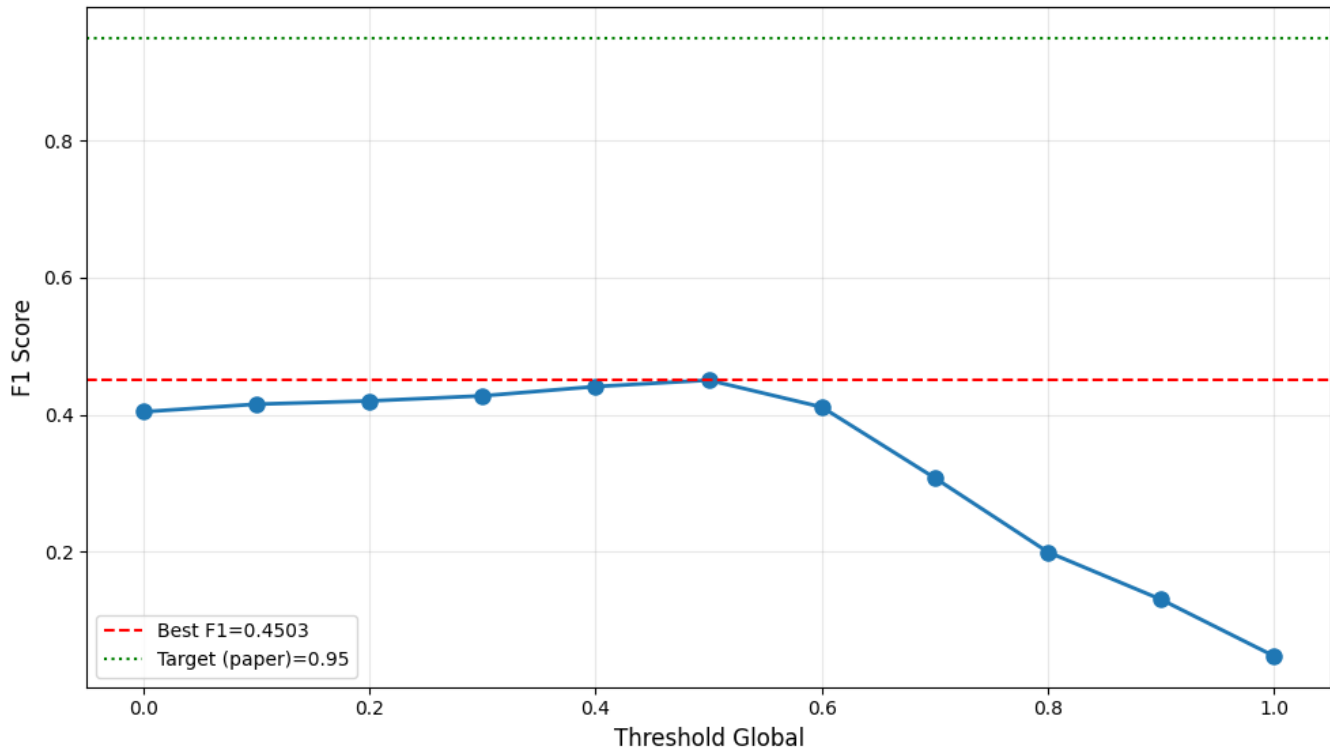
```

```

=====
BARRIDO DE THRESHOLDS - F1 POR THRESHOLD
=====
t=0.0: F1=0.4041
t=0.1: F1=0.4152
t=0.2: F1=0.4200
t=0.3: F1=0.4274
t=0.4: F1=0.4410
t=0.5: F1=0.4503 ← MEJOR
t=0.6: F1=0.4110
t=0.7: F1=0.3073
t=0.8: F1=0.1990
t=0.9: F1=0.1298
t=1.0: F1=0.0470
=====

```

Board Reconstruction: Barrido Global de Thresholds



Threshold óptimo: $t=0.5$

Reconstruction Score: 0.4503

```
# Estadísticas sobre features de alta precisión por BSP
n_features_per_bsp = [len(features) for features in high_precision_features.values()]

print("="*60)
print("ESTADÍSTICAS - FEATURES POR BSP")
print("="*60)
print(f"Total BSPs: {len(high_precision_features)}")
print(f"Features promedio por BSP: {np.mean(n_features_per_bsp):.1f}")
print(f"Features mediana por BSP: {np.median(n_features_per_bsp):.1f}")
print(f"BSPs con 0 features: {np.sum(np.array(n_features_per_bsp) == 0)}")
print(f"BSPs con 1+ features: {np.sum(np.array(n_features_per_bsp) > 0)}")
print(f"BSPs con 5+ features: {np.sum(np.array(n_features_per_bsp) >= 5)}")
print(f"BSPs con 10+ features: {np.sum(np.array(n_features_per_bsp) >= 10)}")
print(f"Max features en una BSP: {np.max(n_features_per_bsp)}")
print("="*60)
```

```
=====
ESTADÍSTICAS - FEATURES POR BSP
=====

Total BSPs: 128
Features promedio por BSP: 1920.8
Features mediana por BSP: 1845.0
BSPs con 0 features: 0
BSPs con 1+ features: 128
BSPs con 5+ features: 128
BSPs con 10+ features: 128
Max features en una BSP: 4630
=====
```

```
# Comparación con resultados del paper
print("="*60)
print("COMPARACIÓN CON PAPER (Karvonen et al., NeurIPS 2024)")
print("="*60)
print(f"{'Modelo':<20} {'Reconstruction':<15}")
print("-"*60)
print(f"{'SAE random':<20} {'0.08':<15}")
print(f"{'SAE trained (paper)':<20} {'0.95':<15} ← Objetivo")
print(f"{'Linear probe':<20} {'0.99':<15}")
print(f"{'Nuestro SAE':<20} {reconstruction_score:<15.4f} ← Resultado")
print("="*60)

if reconstruction_score >= 0.90:
    print("\n Excelente: Muy cercano al objetivo del paper")
elif reconstruction_score >= 0.80:
    print("\n Bueno: Por encima de random pero debajo del objetivo")
else:
    print("\n Bajo: Revisar implementación o entrenamiento del SAE")
```

```
=====
COMPARACIÓN CON PAPER (Karvonen et al., NeurIPS 2024)
=====
Modelo                Reconstruction
-----
SAE random            0.08
SAE trained (paper)   0.95                ← Objetivo
Linear probe          0.99
Nuestro SAE           0.4503            ← Resultado
=====
```

Bajo: Revisar implementación o entrenamiento del SAE

0.9 9. Guardar Resultados

```
# Guardar resultados
results = {
    'reconstruction_score': reconstruction_score,
    'best_threshold': best_threshold,
    'f1_per_threshold': f1_per_threshold,
    'global_thresholds': np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]),
    'phase1_time_seconds': phase1_time,
    'phase2_time_seconds': phase2_time,
    'total_time_seconds': total_time,
    'n_features_per_bsp': n_features_per_bsp,
    'bsp_names': bsp_pieces_names
}

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos']
)
output_path = metrics_dir / "reconstruction_results.npz"
np.savez(output_path, **results)

print(f" Resultados guardados en:")
print(f" {output_path}")
print(f"\nContenido guardado:")
```

```

print(f" - reconstruction_score: {reconstruction_score:.4f}")
print(f" - best_threshold: {best_threshold:.1f}")
print(f" - f1_per_threshold: array de {len(f1_per_threshold)} valores")
print(f" - n_features_per_bsp: array de {len(n_features_per_bsp)} valores")

```

Resultados guardados en:

c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1_standard\1000games\metrics

Contenido guardado:

- reconstruction_score: 0.4503
- best_threshold: 0.5
- f1_per_threshold: array de 11 valores
- n_features_per_bsp: array de 128 valores

```

import subprocess
from sae.tools.experiment_utils import get_report_name

output_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos']
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'reconstruction'
)

notebook_name = 'reconstruction_gpu_optimized.ipynb'
output_path = output_dir / pdf_name

print(f' Generando PDF con Quarto...')
print(f' Archivo de salida: {output_path}')

result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{output_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f' PDF generado exitosamente: {output_path}')
else:
    print(f' Error al generar PDF:')
    print(result.stderr)

```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1_standard

PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1